20 van Emde Boas 樹 解答

20-E01 bitmap successor

推導

位元為 1 的位置是 2、5、6。successor(2)=5；predecessor(6)=5；min=2。位置 0 左起最左位元，題目先定義位序以免方向歧義。

結論與檢核

整數 universe 的答案不需掃描 key 集合；bitmap 可直接用位運算定位。若查詢大於 vEB 最大值，successor 回傳 NIL。

20-E02 cluster index

推導

cluster=floor(173/16)=10，low=173 mod 16=13。因此 key 位於 cluster[10] 的局部鍵 13。

結論與檢核

再以 cluster[10].min/max 與 13 比較，可判斷 successor 是否仍在 cluster。若173不是本層獨立保存的min，便存入cluster10局部13，summary包含10。若集合只有173，min=max=173，所有cluster與summary都空；高低索引本身不代表該鍵一定重複存入 cluster。

20-E03 vEB 遞迴深度

推導

初始 lg u=32。cluster universe 大小為 sqrt(u)，lg 從 32 變 16，再變 8、4、2、1，約六層。

結論與檢核

所以量級為$O(\lg \lg u)$，本題約六層，而不是32或2^{32}。這是6次節點呼叫、5次遞迴下降；若每層兩個昂貴呼叫，僅計深度便不成立。原版預先配置的空間仍Theta(u)，不由這個時間深度變成Theta(lg lg u)。

20 van Emde Boas 樹 解答 （續）

20-E04 Min不進Cluster的更新追蹤

推導

原min2/max15，群0空、群1{1,2}、群2{1}、群3{0,3}，summary{1,2,3}。insert1先與min2交換，新min1，原2放群0局部2，summary變{0,1,2,3}。delete1從summary.min=0及群0.min=2取回2成新min，再刪群0局部2；群0空，因此summary刪0，回到{1,2,3}。

結論與檢核

delete15對群3刪局部3後，該群尚有局部0，summary不刪3；新max=$3 \cdot 4 + 0 = 12$。最後集合{2,5,6,9,12}。若把min2同時留在群0，第一次delete1後可能重複代表2，且summary不再等價於『除獨立min外的非空群』，所以會破壞表示不變量。

20-E05 兩次呼叫不一定是兩條昂貴遞迴

推導

插入空群時，cluster只用empty-insert改min/max，O(1)；昂貴遞迴只在summary。非空群只遞迴cluster。刪除後群空，表示刪前群只有一元素，該次cluster刪除已在O(1)單元素基底結束，才會遞迴summary；若cluster刪除昂貴，它不會變空，就不進summary。

結論與檢核

故原版每層最多一條昂貴分支。對可等分b=lg u，proto-minimum有T(u)=2T(sqrt(u))+Theta(1)，換元S(b)=2S(b/2)+Theta(1)，得Theta(lg u)。proto-successor另需一次Theta(lg u)的minimum，S(b)=2S(b/2)+Theta(b)，得Theta(b lg b)=Theta(lg u lg lg u)。單看universe開根號不能推出雙對數。

20-E06 大Universe的初始化不能省略

推導

沿cluster的分群最終把universe分成u/2個大小2的基底，每個至少4bytes，所以僅這些基底就至少$2^{31} \cdot 4 = 2^{33}\ \text{bytes} = 8\ \text{GiB}$，還沒有計summary與其他欄位。這是指定word模型的保守下界，不是某個實作的記憶體實測。

結論與檢核

vEB總時間要計Theta(u)+O(m(1+lg lg u))，比較樹空初態O(1)、操作O(m lg(n+1))、空間O(n)。m不夠大時vEB初始化可主導總成本。不能只比較單次操作的log階數；lazy配置或hash版本是其他設計，必須重新說明配置成本與最壞/期望/攤銷的保證。

20 van Emde Boas 樹 解答 （續）

原創驗算程式附錄: veb_checks.py

```
"""Preallocated vEB implementation with explicit minimum exclusion invariant."""
from random import Random


class VEB:
    def __init__(self, universe):
        assert universe>=2 and universe&(universe-1)==0
        self.u=universe
        self.minimum=self.maximum=None
        if universe>2:
            bits=universe.bit_length()-1
            self.lower=1<<(bits//2)
            self.upper=1<<((bits+1)//2)
            self.summary=VEB(self.upper)
            self.cluster=[VEB(self.lower) for _ in range(self.upper)]

    def member(self,x):
        if self.minimum is None:
            return False
        if x in (self.minimum,self.maximum):
            return True
        return False if self.u==2 else self.cluster[x//self.lower].member(x%self.lower)

    def successor(self,x):
        if self.minimum is None or x>=self.maximum:
            return None
        if x<self.minimum:
            return self.minimum
        if self.u==2:
            return 1
        high,low=divmod(x,self.lower)
        cluster=self.cluster[high]
        if cluster.maximum is not None and low<cluster.maximum:
            return high*self.lower+cluster.successor(low)
        next_cluster=self.summary.successor(high)
        if next_cluster is None:
            return None
        return next_cluster*self.lower+self.cluster[next_cluster].minimum

    def predecessor(self,x):
        if self.minimum is None or x<=self.minimum:
            return None
        if x>self.maximum:
            return self.maximum
        if self.u==2:
            return 0
        high,low=divmod(x,self.lower)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

20 van Emde Boas 樹 解答 （續）

```
        cluster=self.cluster[high]
        if cluster.minimum is not None and low>cluster.minimum:
            return high*self.lower+cluster.predecessor(low)
        previous=self.summary.predecessor(high)
        if previous is None:
            return self.minimum if x>self.minimum else None
        return previous*self.lower+self.cluster[previous].maximum

    def insert(self,x):
        assert 0<=x<self.u
        if self.member(x):
            return False
        self._insert(x)
        return True

    def _insert(self,x):
        if self.minimum is None:
            self.minimum=self.maximum=x
            return
        if x<self.minimum:
            x,self.minimum=self.minimum,x
        if self.u>2:
            high,low=divmod(x,self.lower)
            child=self.cluster[high]
            if child.minimum is None:
                self.summary._insert(high)
                child.minimum=child.maximum=low
            else:
                child._insert(low)
        if x>self.maximum:
            self.maximum=x

    def delete(self,x):
        assert 0<=x<self.u
        if not self.member(x):
            return False
        self._delete(x)
        return True

    def _delete(self,x):
        if self.minimum==self.maximum:
            self.minimum=self.maximum=None
            return
        if self.u==2:
            self.minimum=self.maximum=1 if x==0 else 0
            return
        if x==self.minimum:
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

20 van Emde Boas 樹 解答 （續）

```
            first=self.summary.minimum
            x=first*self.lower+self.cluster[first].minimum
            self.minimum=x
        high,low=divmod(x,self.lower)
        child=self.cluster[high]
        child._delete(low)
        if child.minimum is None:
            self.summary._delete(high)
            if x==self.maximum:
                last=self.summary.maximum
                self.maximum=self.minimum if last is None else
last*self.lower+self.cluster[last].maximum
        elif x==self.maximum:
            self.maximum=high*self.lower+child.maximum

    def validate(self):
        if self.u==2:
            assert (self.minimum is None)==(self.maximum is None)
            if self.minimum is None:
                return set()
            assert 0<=self.minimum<=self.maximum<2
            return {self.minimum,self.maximum}
        summary=self.summary.validate()
        values=set()
        nonempty=set()
        for high,child in enumerate(self.cluster):
            local=child.validate()
            if local:
                nonempty.add(high)
            values.update(high*self.lower+low for low in local)
        assert summary==nonempty
        if self.minimum is None:
            assert not values and self.maximum is None
            return set()
        assert self.minimum not in values
        values.add(self.minimum)
        assert min(values)==self.minimum and max(values)==self.maximum
        return values


def check_queries(tree, expected):
    assert tree.validate()==expected
    for x in range(tree.u):
        assert tree.member(x)==(x in expected)
        assert tree.successor(x)==min((y for y in expected if y>x),default=None)
        assert tree.predecessor(x)==max((y for y in expected if y<x),default=None)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

20 van Emde Boas 樹 解答 （續）

```
def main():
    tree=VEB(16)
    for key in (2,5,6,9,12,15):
        tree.insert(key)
    check_queries(tree,{2,5,6,9,12,15})
    assert tree.summary.validate()=={1,2,3}
    assert tree.cluster[0].minimum is None
    assert (tree.successor(5),tree.successor(6),tree.predecessor(5))==(6,9,2)
    tree.insert(1)
    assert tree.cluster[0].validate()=={2}
    tree.delete(1)
    assert tree.cluster[0].minimum is None and tree.minimum==2
    tree.delete(15)
    assert tree.maximum==12
    for mask in range(256):
        tree=VEB(8)
        expected={x for x in range(8) if mask&(1<<x)}
        for x in sorted(expected,reverse=True):
            tree.insert(x)
        check_queries(tree,expected)
        for x in range(8):
            assert tree.delete(x)==(x in expected)
            expected.discard(x)
            check_queries(tree,expected)
    rng=Random(20)
    operations=0
    for universe in (2,4,8,16,32,64,256):
        tree=VEB(universe)
        expected=set()
        for _ in range(400):
            x=rng.randrange(universe)
            if rng.randrange(2):
                assert tree.insert(x)==(x not in expected)
                expected.add(x)
            else:
                assert tree.delete(x)==(x in expected)
                expected.discard(x)
            check_queries(tree,expected)
            operations+=1
    print(f"PASS vEB_all_u8_subsets=256 mixed_operations={operations} odd_bit_universes=8,32
min_exclusion_summary_and_all_queries=True")


if __name__=="__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study